

Assignment 2

Architectural Choices, Tokenizers, Decoding, and Fine Tuning

Yehonatan Kapach (318515756)

May 2026

Contents

1	Part 1 - Architectural Choices	3
1.1	Extraction Methodology	3
1.2	Per-Model Architectural Details	3
1.2.1	meta-llama/Llama-3.1-8B-Instruct	3
1.2.2	mistralai/Mistral-7B-Instruct-v0.3	3
1.2.3	Qwen/Qwen2.5-7B-Instruct	4
1.2.4	allenai/OLMo-2-1124-7B-Instruct	5
1.2.5	ibm-granite/granite-3.3-8b-instruct	5
1.2.6	deepseek-ai/DeepSeek-V3	6
1.2.7	HuggingFaceTB/SmolLM2-1.7B-Instruct	7
1.2.8	microsoft/Phi-4-mini-instruct	7
1.2.9	tiiuae/Falcon3-7B-Instruct	8
1.2.10	dicta-ll/dictalm2.0-instruct	9
1.3	Uncertainties, Missing Fields, and Source Disagreements	9
1.4	Architectural Comparison	10
1.5	Reflection and Analysis	11
2	Part 2 - Tokenizers	12
2.1	Tokenizer Choices per Model	12
2.2	Average Tokens per Word	13
2.3	Tokenization Differences Example	13
2.4	Discussion	14
3	Part 3 - Constrained Decoding	16
3.1	Hebrew Token Identification	16
3.2	Implementation	16
3.3	Results and Discussion	17
4	Part 4 - Fine Tuning	19
4.1	Data	19

4.2	Fine-Tuning Procedure	19
4.3	Evaluation	19

1 Part 1 - Architectural Choices

1.1 Extraction Methodology

Architectural information for each model was gathered using **NotebookLM**; I ran a deep research experiment to collect sources for all models. After gathering a reliable list of sources, I asked **NotebookLM** to retrieve the relevant information for each model, while including references to the sources.

1.2 Per-Model Architectural Details

1.2.1 meta-llama/Llama-3.1-8B-Instruct

1. Layers and width. 32 hidden layers^[1]. The hidden size is described as “typically 4096–5120” without specifying the exact value for the 8B variant^[1]; see Section 1.3.

2. Attention heads. 32 query heads per layer^[1]. The model uses Grouped-Query Attention (GQA)^[1,2]; the exact number of KV heads is not reported in the available sources (see Section 1.3).

3. Component sizes. Dense decoder-only architecture; no MoE layers^[1]. Exact MLP intermediate size, attention head dimensions, and vocabulary and embedding matrix sizes are all absent from the sources.

4. Position encoding. RoPE^[1], extended via the *llama3* multi-scale scheme^[3]. Context length: 128k tokens^[2] (Source 1 states the tokenizer supports up to 8,192 tokens - disagreement noted in Section 1.3). Scaling hyper-parameters: factor = 8.0, low-freq factor = 1.0, high-freq factor = 4.0, original max positions = 8,192, `rope_type = llama3`^[3].

5. Activations. SwiGLU, uniformly across all feed-forward sublayers^[1].

6. Normalization. Pre-norm placement^[1]. Sources describe the norm type as “RMSNorm or LayerNorm” without definitively specifying one^[1]; RMSNorm is standard for this model family.

7. Other properties. ≈ 8.03 B total parameters^[1]. Instruction-tuned via SFT + DPO/RLHF^[1]. Supports 8 languages natively; knowledge cutoff December 2023^[2]. Training used 1.46M GPU-hours on H100-80 GB hardware, producing ≈ 420 t CO₂eq^[2].

Sources:

[1] <https://www.emergentmind.com/topics/llama-3-1-8b-instruct-679bb9c6-400c-4f7c-b10a-c1048d659134>

[2] <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>

[3] <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct/discussions/15>

1.2.2 mistralai/Mistral-7B-Instruct-v0.3

1. Layers and width. 32 hidden layers^[2,3]; hidden size 4,096^[2,3].

2. Attention heads. 32 query heads, 8 KV heads (GQA, group size 4)^[2,3]. Faster inference via GQA is highlighted in^[1,2].

3. Component sizes. Dense decoder; no MoE. MLP intermediate size: 14,336 ($\approx 3.5 \times$ hidden)^[2,3]. Attention head dimension: 128^[2]. Vocabulary size: 32,768 (extended from 32,000 in v0.1/v0.2)^[3,4]. Separate (untied) input and output embedding matrices; exact sizes inferred from hidden size and vocabulary.

4. Position encoding. RoPE^[2]; $\theta = 1,000,000$; maximum context 32,768 tokens^[3]. Sliding Window Attention (window = 4,096) is architecturally supported^[1,2], but disabled (`sliding_window: null`) in the v0.3 configuration^[3].

5. Activations. SiLU, uniformly across all MLP layers^[3].

6. Normalization. RMSNorm, $\varepsilon = 10^{-5}$ ^[3]. Exact placement (pre- vs. post-norm) not stated in the sources; pre-norm is the standard for this family.

7. Other properties. ≈ 7.3 B parameters^[1]. v0.3 release adds the v3 tokenizer and native function calling^[4]. Apache 2.0 license^[1,3,4].

Sources:

[1] <https://mistral.ai/news/announcing-mistral-7b>

[2] <https://www.emergentmind.com/papers/mistral-7b-instruct>

[3] <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3/blob/main/config.json>

[4] <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>

1.2.3 Qwen/Qwen2.5-7B-Instruct

1. Layers and width. 28 hidden layers^[1,2]; hidden size 3,584^[2].

2. Attention heads. 28 query heads, 4 KV heads (GQA, group size 7)^[1,2].

3. Component sizes. Dense decoder; no MoE. MLP intermediate size: 18,944 ($\approx 5.3 \times$ hidden) - notably wider than peers^[2]. Attention head dimension not explicitly stated; inferred as 128 (3584/28). Vocabulary size: 152,064^[2]. 7.61B total parameters (6.53B non-embedding)^[1]. Untied input/output embeddings.

4. Position encoding. RoPE with $\theta = 1,000,000$ ^[2]. Base context: 32,768 tokens^[2]; extended to 131,072 via YaRN^[1] (type=yarn, factor=4.0, original_max_position_embeddings=32,768^[1]). Generation is limited to 8,192 output tokens^[1].

5. Activations. SiLU (`hidden_act: silu`), uniformly^[1,2].

6. Normalization. RMSNorm, $\varepsilon = 10^{-6}$ ^[1,2]. Exact placement not stated in the sources.

7. Other properties. Uses **QKV bias** in attention projections - uncommon among modern models^[1]. Supports 29+ languages^[1]. Apache 2.0 license^[1].

Sources:

[1] <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>

[2] <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct/blob/main/config.json>

1.2.4 allenai/OLMo-2-1124-7B-Instruct

1. Layers and width. 32 hidden layers; hidden size 4,096^[1].

2. Attention heads. 32 query heads and 32 KV heads - full Multi-Head Attention (MHA), no KV-head reduction^[1].

3. Component sizes. Dense decoder; no MoE. MLP intermediate size: 11,008 ($\approx 2.7 \times$ hidden)^[1]. Head dimension inferred as 128 (4096/32). Vocabulary size: 100,352; embeddings not tied (`tie_word_embeddings: false`)^[1]. No attention bias and no MLP bias^[1].

4. Position encoding. RoPE, $\theta = 500,000$ ^[1]. Maximum context: 4,096 tokens; `rope_scaling` set to null - no additional scaling applied^[1].

5. Activations. SiLU, uniformly^[1].

6. Normalization. RMSNorm, $\varepsilon = 10^{-6}$ ^[1]. Exact placement not stated in the sources.

7. Other properties. Post-trained via SFT + DPO + RLVR on the T-lu 3 dataset^[2]. Base model pre-trained on the Dolma dataset^[2]. All code, checkpoints, and training logs are publicly released^[2].

Sources:

[1] <https://huggingface.co/allenai/OLMo-2-1124-7B-Instruct/blob/main/config.json>

[2] <https://huggingface.co/eaddario/OLMo-2-1124-7B-Instruct-GGUF>

1.2.5 ibm-granite/granite-3.3-8b-instruct

*Note: the instruct model shares the network geometry of its base model **granite-3.3-8b-base**; parameters below are extracted from base model documentation^[1].*

1. Layers and width. 40 hidden layers; hidden size 4,096^[1].

2. Attention heads. 32 query heads, 8 KV heads (GQA, group size 4)^[1].

3. Component sizes. Dense decoder; no MoE^[1]. MLP intermediate size: 12,800 ($\approx 3.1 \times$ hidden)^[1]. Attention head dimension: 128^[1]. Shared input/output embeddings (tied)^[1]; exact vocabulary size is not reported in the sources.

4. Position encoding. RoPE; maximum context 128,000 tokens^[1,2,3]. Base θ and any scaling factors are not stated in the sources.

5. Activations. SwiGLU, uniformly across all layers^[1].

6. Normalization. RMSNorm^[1]. Exact placement and ε not stated in the sources.

7. Other properties. 8.1B total parameters, all active at inference^[1]. Pretrained on 12T tokens^[1]. Natively supports `<think>/<response>` tags for structured reasoning^[2] and Fill-in-the-Middle (FIM) code completion^[1]. 12 supported languages^[2].

Sources:

[1] <https://huggingface.co/ibm-granite/granite-3.3-8b-base>

[2] <https://build.nvidia.com/ibm/granite-3-3-8b-instruct>

[3] <https://navigator.ufl.edu/>

1.2.6 deepseek-ai/DeepSeek-V3

1. Layers and width. 61 Transformer layers^[2,3] (first 3 are dense FFN; the remaining 58 are MoE)^[3]. Hidden dimension: 7,168^[1,3].

2. Attention heads. 128 query heads^[3]. The model uses Multi-Head Latent Attention (MLA) instead of standard MHA/GQA: keys and values are compressed into a shared 512-dimensional latent vector ($d_c = 512$), then decompressed per-head, dramatically reducing KV cache memory^[3,5]. Query compression latent: $d'_c = 1,536$ ^[3]. Attention head dimension: $d_h = 128$ ^[3].

3. Component sizes. Sparse MoE decoder. Per MoE layer: 257 total experts - 1 shared expert (always active) plus 256 routed experts^[2,3]. Per token: 9 experts activated (1 shared + 8 routed)^[2,3]. Per-expert intermediate dimension: 2,048; combined active dimension: $9 \times 2048 = 18,432$ ^[1,3]. Decoupled RoPE dimension per head: $d_h^R = 64$ ^[3]. Shared input and output embedding matrices^[3]. Vocabulary size: 128,000^[3] (Source 1 lists 129,280 - see Section 1.3).

4. Position encoding. Decoupled RoPE embedded within MLA: positional information is concatenated to the KV representations rather than applied directly, resolving a mathematical incompatibility with MLA compression^[3,5]. Maximum context: 131,072 tokens (128k)^[3,4]. Context extended from an initial 4k via YaRN: $s = 40$, $\alpha = 1$, $\beta = 32$, scaling factor $t = 0.1 \ln s + 1$ ^[3].

5. Activations. SwiGLU/SiLU, used consistently across all MoE and dense layers^[1,2].

6. Normalization. RMSNorm, pre-norm placement^[1,2,3]. Additionally, extra RMSNorm layers are inserted *after* the compressed MLA latent vectors - a consequence of the MLA architecture^[3].

7. Other properties. 671B total parameters; 37B active per token (4.8%)^[2,3,4,6]. Auxiliary-loss-free load balancing: a dynamic routing bias prevents collapse without penalising the main loss^[3,6]. Multi-Token Prediction (MTP) training objective: predicts 2 tokens per step (depth $D = 1$)^[3,5]. First large-scale model pre-trained natively in FP8 mixed precision; full training cost $\approx \$5.58\text{M}$ on H800 GPUs^[2,3,6].

Sources:

- [1] <https://www.atlascloud.ai/blog/guides/analyzing-deepseek-v3-model-performance>
- [2] <https://fireworks.ai/blog/deepseek-model-architecture>
- [3] <https://arxiv.org/abs/2412.19437>
- [4] <https://github.com/deepseek-ai/DeepSeek-V3>
- [5] <https://mccormickml.com/2025/02/12/the-inner-workings-of-deep-seek-v3/>
- [6] <https://www.grandlinux.com/en/blogs/deepseek-moe-architecture.html>

1.2.7 HuggingFaceTB/SmolLM2-1.7B-Instruct

- 1. Layers and width.** 24 hidden layers; hidden size 2,048^[1].
- 2. Attention heads.** 32 query heads and 32 KV heads - full MHA, no KV reduction^[1].
- 3. Component sizes.** Dense decoder (LlamaForCausalLM architecture); no MoE^[1]. MLP intermediate size: 8,192 ($4 \times$ hidden)^[1]. Head dimension inferred as 64 (2048/32). Vocabulary size: 49,152^[1,2]. Tied word embeddings (`tie_word_embeddings: true`)^[1,2].
- 4. Position encoding.** RoPE, $\theta = 130,000$; `rope_scaling` is `null` (no additional scaling)^[1]. Maximum context: 8,192 tokens per `config.json`^[1]; a secondary source lists 2,048 tokens^[2] - see Section 1.3.
- 5. Activations.** SiLU (`hidden_act: silu`), uniformly^[1].
- 6. Normalization.** RMSNorm, $\varepsilon = 10^{-5}$ ^[1]. Exact placement not stated in the sources.
- 7. Other properties.** 1.7B parameters^[2,3]; designed for edge, mobile, and browser deployment. Pretrained on 11T tokens (FineWeb-Edu, DCLM, The Stack, Cosmopedia v2, math and coding datasets)^[2,3]. Post-trained via SFT + DPO on UltraFeedback^[3]. Training used 256 NVIDIA H100 GPUs in bfloat16^[2,3].

Sources:

- [1] <https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B-Instruct/blob/main/config.json>
- [2] <https://collabnix.com/hugging-face-small-language-model-a-complete-guide/>
- [3] <https://mirai.us/models/SmolLM2-1.7B-Instruct>

1.2.8 microsoft/Phi-4-mini-instruct

- 1. Layers and width.** 32 hidden layers; hidden size 3,072^[2].
- 2. Attention heads.** 24 query heads, 8 KV heads (GQA, group size 3)^[2,3,4].

3. Component sizes. Dense decoder; no MoE^[1,3,4]. MLP intermediate size: 8,192 ($\approx 2.7 \times$ hidden)^[2]. Head dimension inferred as 128 (3072/24). Vocabulary size: 200,064^[2,3,4]. Tied word embeddings (`tie_word_embeddings: true`)^[1,2,3,4]. No attention or MLP biases anywhere^[2].

4. Position encoding. LongRoPE (`type: longrope`)^[2]. Base $\theta = 10,000$; original max positions: 4,096^[2]. Extended to 131,072 tokens^[1,3,4] via a `rope_scaling` dictionary containing a `short_factor` array (all 1.0) and a `long_factor` array with 48 dynamically scaled multipliers (ranging from 1.0 to 47.77)^[2]. Partial rotary factor: 0.75 - RoPE is applied to only 75% of each head's dimensions^[2].

5. Activations. SiLU (`hidden_act: silu`), uniformly^[2].

6. Normalization. RMSNorm, $\varepsilon = 10^{-5}$ ^[2]. Exact placement not stated in the sources.

7. Other properties. 3.8B parameters^[1,3,4]. Trained on 5T tokens (high-quality educational data, code, synthetic data, filtered web text); knowledge cutoff June 2024^[3,4]. Post-trained via SFT + DPO + RLHF^[1,3,4].

Sources:

- [1] <https://catalog.models.ai.microsoft.com/>
- [2] <https://huggingface.co/microsoft/Phi-4-mini-instruct/blob/main/config.json>
- [3] <https://build.nvidia.com/microsoft/phi-4-mini-instruct>
- [4] <https://huggingface.co/microsoft/Phi-4-mini-instruct>

1.2.9 tiiuae/Falcon3-7B-Instruct

1. Layers and width. 28 hidden layers^[1,2]; hidden size 3,072^[1].

2. Attention heads. 12 query heads, 4 KV heads (GQA, group size 3)^[1,2]. Attention head dimension: 256^[1,2] - double the 128 used by all other models in this set.

3. Component sizes. Dense decoder (LlamaForCausalLM); no MoE^[1,2]. MLP intermediate size: 23,040 ($\approx 7.5 \times$ hidden - largest ratio in this set)^[1]. Vocabulary size: 131,072; untied embeddings (`tie_word_embeddings: false`)^[1]. No attention or MLP biases^[1].

4. Position encoding. RoPE, $\theta = 1,000,042$ ^[1,2]; `rope_scaling = null`^[1]. Maximum context: 32,768 tokens^[1] (Source 2 summarises as “32k”).

5. Activations. SwiGLU (`hidden_act: silu`), uniformly^[1,2].

6. Normalization. RMSNorm, $\varepsilon = 10^{-6}$ ^[1,2]. Exact placement not stated in the sources.

7. Other properties. Despite the “Falcon” brand, the code architecture class is LlamaForCausalLM^[1]. Pretrained on 14 teratokens; post-trained on 1.2M samples covering conversation, code, STEM, safety, and function calling^[2]. Falcon-LLM-License (not Apache 2.0)^[1,2].

Sources:

[1] <https://huggingface.co/tiiuae/Falcon3-7B-Instruct/blob/main/config.json>

[2] <https://build.nvidia.com/tiiuae/falcon3-7b-instruct>

1.2.10 dicta-il/dictalm2.0-instruct

1. Layers and width. 32 hidden layers; hidden size 4,096^[2].

2. Attention heads. 32 query heads, 8 KV heads (GQA, group size 4)^[2].

3. Component sizes. Dense decoder based on Mistral-7B-v0.1; no MoE^[2,3]. MLP intermediate size: 14,336 ($\approx 3.5 \times$ hidden)^[2]. Attention head dimension (RoPE dimension count): 128^[2]. Vocabulary size: 33,152^[2]. Separate (untied) input embedding and output matrices, each of size $4096 \times 33,152$ ^[2].

4. Position encoding. RoPE; base frequency (θ): 10,000; applied over 128 dimensions^[2]. Maximum context: 32,768 tokens^[1,2].

5. Activations. Activation function not explicitly named in the sources. Inferred as SiLU/SwiGLU from the presence of `ffn_gate`, `ffn_up`, and `ffn_down` weight matrices^[2], consistent with the Mistral-7B-v0.1 base architecture^[3].

6. Normalization. RMSNorm, $\varepsilon = 10^{-5}$ ^[2]. Pre-norm placement is confirmed by the presence of `attn_norm`, `ffn_norm`, and `output_norm` weight tensors in the model blobs^[2].

7. Other properties. The only Hebrew-specialised model in this set. Built on Mistral-7B-v0.1, then continually pre-trained on 190B tokens (50% Hebrew, 50% English)^[3]. Hebrew tokenisation rate improved from ≈ 5.78 to **2.76 tokens/word** through the injection of $\approx 1,000$ Hebrew-specific tokens into the vocabulary^[3].

Sources:

[1] <https://ollama.com/aminadaven/dictalm2.0-instruct>

[2] https://ollama.com/aminadaven/dictalm2.0-instruct:q4_0

[3] <https://huggingface.co/dicta-il/dictalm2.0>

1.3 Uncertainties, Missing Fields, and Source Disagreements

Llama Sources^[1] describe the hidden size as “typically 4096–5120” without pinning the 8B model to a specific value; 4096 is used as the well-known value for this variant. KV head count is absent from all three sources; 8 is the accepted value. Norm type is listed as “RMSNorm *or* LayerNorm”^[1] without resolution. Source 1 states the tokenizer supports 8,192 tokens, while Source 2 lists the context length as 128k; the latter is treated as authoritative for the `max_position_embeddings` configuration.

DeepSeek-V3 Vocabulary size: the arXiv technical report^[3] states 128,000, while an Atlas Cloud analysis^[1] lists 129,280. Source 3 is treated as authoritative. The MLP “size” of 2,048 refers to each individual expert’s intermediate dimension; the effective combined dimension across all 9 active experts is 18,432, which is the figure Source 1 reports as the MLP dimension.

- Granite** Vocabulary size is not reported in any of the three sources consulted. RoPE θ value and any scaling parameters are also absent; these fields are left as NA.
- SmolLM2** Source 2 states the context length as 2,048 tokens, while `config.json`^[1] sets `max_position_embeddings` to 8,192. The configuration file is treated as authoritative; the discrepancy likely reflects documentation lag or a deployment-level truncation.

1.4 Architectural Comparison

Table 1: Architectural parameters - all 10 models

Model	Lay.	Hidden	Q-h	KV-h	MLP	Ctx (k)	Vocab
Llama-3.1-8B	32	4096	32	8	14336	128	128k
Mistral-7B v0.3	32	4096	32	8	14336	32	33k
Qwen2.5-7B	28	3584	28	4	18944	128	152k
OLMo-2-7B	32	4096	32	32	11008	4	100k
Granite-3.3-8B	40	4096	32	8	12800	128	NA
DeepSeek-V3	61	7168	128	MLA	2048 (per exp.)	128	128k
SmolLM2-1.7B	24	2048	32	32	8192	8	49k
Phi-4-mini	32	3072	24	8	8192	128	200k
Falcon3-7B	28	3072	12	4	23040	32	131k
DictaLM2.0	32	4096	32	8	14336	32	33k

Table 2: Activation, norm, and position-encoding scheme per model

Model	Act.	Norm	Position encoding
Llama-3.1-8B	SwiGLU	RMSNorm	RoPE, llama3 multi-scale scaling
Mistral-7B v0.3	SiLU	RMSNorm	RoPE, $\theta=1\text{M}$; SWA disabled
Qwen2.5-7B	SiLU	RMSNorm	RoPE + YaRN (factor 4.0)
OLMo-2-7B	SiLU	RMSNorm	RoPE, $\theta=500\text{k}$, no scaling
Granite-3.3-8B	SwiGLU	RMSNorm	RoPE, 128k ctx (params unknown)
DeepSeek-V3	SwiGLU	RMSNorm	Decoupled RoPE + YaRN in MLA
SmolLM2-1.7B	SiLU	RMSNorm	RoPE, $\theta=130\text{k}$, no scaling
Phi-4-mini	SiLU	RMSNorm	LongRoPE, partial rotation (0.75)
Falcon3-7B	SwiGLU	RMSNorm	RoPE, $\theta=1\text{M}$, head_dim=256
DictaLM2.0	SiLU	RMSNorm	RoPE, $\theta=10\text{k}$ (lowest in set)

Attention mechanism. Seven of ten models use GQA (predominantly 8 KV heads for a 4:1 ratio against 32 Q heads). Two models - OLMo-2 and SmolLM2 - retain full MHA (32 KV heads = 32 Q heads), which is viable at their scale where KV cache pressure is lower. DeepSeek-V3 uses MLA, eliminating traditional KV heads by compressing into a 512-dim latent.

MLP width. The intermediate-to-hidden ratio spans from $2.7\times$ (OLMo-2, Phi-4-mini) to $7.5\times$ (Falcon3). The majority cluster at $3.5\times$ (Llama, Mistral, DictaLM). Qwen ($5.3\times$) and Falcon ($7.5\times$) are wide outliers; Falcon compensates for a narrow hidden size and few Q heads by widening the FFN.

Context length strategies. All models use RoPE but differ in how they extend it: a high base θ alone (Mistral: 1M, Falcon: ≈ 1 M), YaRN on top of a high θ (Qwen, DeepSeek), llama3 multi-scale frequency stretching (Llama), and LongRoPE interpolation (Phi). DictaLM’s $\theta=10,000$ is the pre-scaling-era default, making it the weakest at long-context generalisation.

Vocabulary size. Ranges from 33k (Mistral, DictaLM) to 200k (Phi-4-mini). Larger vocabularies reduce tokenisation overhead for code and non-English text at the cost of larger embedding matrices.

1.5 Reflection and Analysis

Consensus choices. RMSNorm with pre-norm, RoPE positional encoding, and a gated linear unit activation (SiLU or SwiGLU) are universal across all ten models. GQA is near-universal for the 7–8B class (7 of 8 applicable models).

Outliers and rules of thumb.

- **Falcon3:** FFN ratio $7.5\times$; head dimension 256 (vs. 128 everywhere else); fewest Q heads (12). A “wide-head” design that compensates narrowness in other dimensions with an unusually large FFN.
- **OLMo-2:** full MHA, $2.7\times$ FFN, 4k context. Prioritises transparency and reproducibility over deployment efficiency.
- **Qwen2.5:** $5.3\times$ FFN, QKV bias, 152k vocab. Heavy investment in multilingual and reasoning performance.
- **DeepSeek-V3:** qualitatively different class - MoE with 671B total / 37B active, MLA attention, FP8 training.
- **DictaLM2.0:** $\theta=10,000$ is the pre-high- θ Mistral default, leaving it the least robust for long-sequence inputs.
- The most common Q/KV ratio among GQA models is 4:1 (32Q/8KV at hidden=4096). Qwen uses 7:1; Falcon and Phi use 3:1.
- Deeper stack (40 layers) for Granite; MoE depth (61 layers) for DeepSeek, where active parameter count per forward pass remains tractable.

Recommended design for a new 7–8B model. Based on the consensus above: 32 layers, hidden size 4,096, GQA with 32Q/8KV heads and head dimension 128. SwiGLU MLP with intermediate size $\approx 14\text{k}–16\text{k}$ ($3.5–4\times$ hidden). RoPE with $\theta \approx 500\text{k}–1\text{M}$; YaRN can be applied at inference time if contexts $> 32\text{k}$ are needed, avoiding baking in complex scaling during pre-training. RMSNorm pre-norm, $\varepsilon = 10^{-5}$. Vocab $\approx 100\text{k}–130\text{k}$ for broad multilingual coverage; tied embeddings only if the model is small or memory-constrained. Avoid MoE and MLA unless the deployment scenario explicitly requires them.

2 Part 2 - Tokenizers

2.1 Tokenizer Choices per Model

Table 3 summarises the three tokenizer properties examined for each model: vocabulary size, word-boundary strategy, and non-word tokens.

Table 3: Tokenizer summary for all 10 models

Model	Class	Vocab	Boundary	Byte	Special
Llama-3.1-8B	PreTrainedTokenizerFast	28,256	Ġ (BPE)	No	2
Mistral-7B	LlamaTokenizerFast	32,768	— (Sentence-Piece)	Yes	3
Qwen2.5-7B	Qwen2TokenizerFast	151,665	Ġ (BPE)	No	14
OLMo-2-7B	GPT2TokenizerFast	100,278	Ġ (BPE)	No	2
Granite-3.3-8B	GPT2TokenizerFast	49,159	Ġ (BPE)	No	8
DeepSeek-V3	LlamaTokenizerFast	128,815	Ġ (BPE)	No	2
SmolLM2-1.7B	GPT2TokenizerFast	49,152	Ġ (BPE)	No	3
Phi-4-mini	GPT2TokenizerFast	200,029	Ġ (BPE)	No	1
Falcon3-7B	PreTrainedTokenizerFast	31,072	Ġ (BPE)	No	25
DictaLM-2.0	LlamaTokenizerFast	33,152	— (Sentence-Piece)	Yes	3

Vocabulary size. Vocabulary sizes span nearly an order of magnitude, from 32,768 tokens (Mistral-7B, DictaLM-2.0) to 200,029 (Phi-4-mini), reflecting different trade-offs between coverage, embedding-table size, and average sequence length.

Word-boundary strategy. All ten tokenizers use a *prefix-marker* convention to encode which tokens are the beginning of a new word, so that the original text can be reconstructed unambiguously from the token sequence alone. Eight models use **GPT-2-style BPE**: a leading Ġ (U+0120) signals that the token opens a new word; tokens without this prefix are continuations that attach directly to the preceding token. The two exceptions—**Mistral-7B** and **DictaLM-2.0**—use **SentencePiece**, which treats the marker —(U+2581) as a literal whitespace placeholder. Reconstruction is straightforward in both cases through left-to-right concatenation. However, while the GPT-2 BPE marker dictates logical word boundaries that must be swapped for spaces, SentencePiece treats its marker as an actual character, allowing for a direct substitution back into standard whitespaces.

Special tokens. The “Special” column counts tokens that encode structural or control information rather than text content: sequence delimiters (<s>, </s>, <|endoftext|>), unknown-token placeholders (<unk>), and chat-format markers. Most models have 1–3 such tokens. Three models stand out:

- **Qwen2.5** (14): adds vision/tool-use delimiters (<|im_start|>, <|im_end|>, <|object_ref_start|>, <|box_start|>, ...).
- **Granite-3.3** (8): adds role-marking tokens (<|start_of_role|>, <|end_of_role|>, <|tool_call|>, <|start_of_cite|>, ...).
- **Falcon3** (25): inherits legacy corpus-metadata markers from pre-training (»TITLE«, »ABSTRACT«, ...).

Byte-fallback tokens. Only the two SentencePiece tokenizers (Mistral, DictaLM) carry an additional 256 <0xNN> byte tokens, one per possible byte value. These are also non-word tokens:

they do not represent text directly but serve as a universal fallback, encoding any Unicode codepoint byte-by-byte whenever it lacks a dedicated vocabulary entry. This guarantees that the tokenizer is losslessly invertible on arbitrary input.

2.2 Average Tokens per Word

Methodology. Two reference corpora were used: a $\sim 1,000$ -word English text and a $\sim 1,000$ -word Hebrew text (both stored in `docs/`). For each model, the corpus was encoded with `tok.encode(text, add_special_tokens=False)`; the *word count* was defined as the number of whitespace-delimited tokens (`len(text.split())`). Average tokens per word is therefore $\frac{\text{encoded token count}}{\text{whitespace word count}}$.

This method assumes that whitespace splitting is a reasonable word-count proxy for both English and Hebrew. Hebrew words are whitespace-delimited (unlike, e.g., Chinese), so the same formula applies directly. The ratio does not account for punctuation attached to words, but this bias is uniform across models and therefore does not affect *comparisons*.

Table 4: Average tokens per word on English and Hebrew corpora

Model	EN avg	HE avg	HE/EN ratio
Llama-3.1-8B	1.29	5.77	4.47
Mistral-7B	1.49	5.97	4.01
Qwen2.5-7B	1.33	2.58	1.94
OLMo-2-7B	1.30	5.78	4.45
Granite-3.3-8B	1.76	5.28	3.00
DeepSeek-V3	1.28	2.86	2.23
SmolLM2-1.7B	1.33	6.39	4.80
Phi-4-mini	1.29	2.42	1.88
Falcon3-7B	1.30	5.31	4.08
DictaLM-2.0	1.49	2.84	1.91

2.3 Tokenization Differences Example

Text used. The following 59-word English passage was used (file `docs/english_text_part2.txt`):

The World Chess Championship 2026 is an upcoming chess match to determine the World Chess Champion. It will be played between the defending champion Gukesh Dommaraju and the challenger, Javokhir Sindarov, the winner of the Candidates Tournament 2026. The Championship is provisionally scheduled between 23 November and 17 December 2026, while the host city is yet to be decided.

The span "Gukesh Dommaraju" (rare South-Indian proper name) was chosen because it produces the greatest variation across tokenizers. Table 5 shows the tokenization by all ten models; four distinct segmentations are produced.

Table 5: Tokenization of "Gukesh Dommaraju" across all 10 models

Model(s)	Tokens
Llama, Qwen, OLMo, DeepSeek, Phi, Falcon	['G', 'uk', 'esh', 'ĠDom', 'mar', 'aju']
Mistral, DictaLM	['_Gu', 'k', 'esh', '—Dom', 'mar', 'aju']
Granite	['G', 'uk', 'esh', 'ĠDom', 'mar', 'aj', 'u']
SmolLM2	['G', 'uk', 'esh', 'ĠD', 'omm', 'ar', 'aj', 'u']

Six of the ten models agree on a single 6-token segmentation. The two SentencePiece models (Mistral and DictaLM) produce the same alternative 6-token split, differing only in the boundary prefix character (— vs. Ġ), which reflects their different tokenizer type rather than their training vocabulary. Granite and SmolLM2 each produce unique segmentations with 7 and 8 tokens respectively, indicating that the substring "Dommaraju" was not frequent enough in their (smaller) training corpora to be represented as a single merged token.

For the common phrase "The World Chess Championship", the majority (6 models: Llama, Qwen, OLMo, DeepSeek, Phi, Falcon) tokenize it into 4 tokens: ['The', 'ĠWorld', 'ĠChess', 'ĠChampionship']. The two SentencePiece models split "Chess" as ['—Che', 'ss'] (5 tokens), Granite fragmentizes "Championship" further into four pieces (8 tokens total), and SmolLM2 splits "Championship" as ['ĠChampions', 'hip'] (5 tokens).

Likely cause. The variation on proper names stems primarily from *vocabulary size and training corpus composition*. Rare names that were seen infrequently receive fewer merged subword units, causing them to be split into smaller fragments. The boundary-marker divergence (Ġ vs. —) is a consequence of the underlying tokenizer algorithm (BPE vs. SentencePiece) and has no effect on reconstruction fidelity.

2.4 Discussion

English efficiency. All models are similarly efficient for English, producing roughly 1.28–1.49 tokens per whitespace word. The outlier is **Granite-3.3** (1.76), which has the smallest base vocabulary (49,159 before special tokens) among the BPE models and therefore covers fewer long English morphemes.

Hebrew efficiency. The results split the ten models into two clear groups:

- **Hebrew-aware** ($HE/EN \leq 3$): Qwen2.5, DeepSeek-V3, Phi-4-mini, DictaLM-2.0. These tokenizers have seen substantial Hebrew during training and have accumulated many whole-Hebrew-word tokens. Phi-4-mini achieves the best ratio ($1.88\times$) with 200,029 vocabulary entries; DictaLM-2.0 benefits from being trained specifically on Hebrew/Israeli text.
- **Hebrew-inefficient** ($HE/EN > 4$): Llama, Mistral, OLMo, SmolLM2, Falcon, and Granite. Hebrew words are fragmented into individual characters or very short byte sequences, producing 5–6 tokens per word. SmolLM2 is the worst at 6.39 tokens per Hebrew word ($4.80\times$ the English rate).

Implications for deployment. A high HE/EN ratio is costly: it inflates context length for Hebrew inputs, reduces effective context window, and increases inference latency and cost. For

a Hebrew-facing application one should prefer Qwen2.5, Phi-4-mini, DeepSeek-V3, or DictaLM-2.0. For purely English tasks the difference is negligible (all models within 0.5 tokens/word of each other).

3 Part 3 - Constrained Decoding

3.1 Hebrew Token Identification

Strategy. For each token ID in a model’s vocabulary we decode it in isolation using `tokenizer.decode([id])` and apply the following rule:

A token is *allowed* if and only if every character in its decoded string that belongs to Unicode letter category (general category starting with L) is also a Hebrew character, i.e. falls in the Unicode Hebrew block U+0590–U+05FF.

This rule keeps: Hebrew letter tokens, digit tokens, all punctuation and whitespace tokens (their characters are never in category L), byte-fallback tokens that decode as the replacement character U+FFFD (their category is So), and special tokens such as EOS/BOS. It rejects: any token containing a Latin, Cyrillic, Arabic, or other non-Hebrew letter. The EOS token is unconditionally added to the allowed set so that generation can always terminate.

The token IDs are written to `hebrew_allowed_tokens_qwen.json` and `hebrew_allowed_tokens_mistral.json`.

Results. Table 6 summarises the allowed-token counts for each model.

Model	Full vocab	Allowed	Hebrew-bearing	Digits/punct/special
Qwen2.5-7B-Instruct	151,665	13,234 (8.7%)	3,164	10,070
Mistral-7B-Instruct-v0.3	32,768	1,912 (5.8%)	38	1,874

Table 6: Hebrew-allowed token counts. “Hebrew-bearing” means the decoded token contains at least one Hebrew character. The remainder are digits, punctuation, whitespace, control, and special tokens that contain no letters.

The large difference in Hebrew-bearing token counts (3,164 vs. 38) reflects training data: Qwen2.5 was trained on multilingual corpora and has tokens for common Hebrew letter sequences and words, while Mistral-7B-v0.3 was trained predominantly on English and European text and retains only 38 Hebrew tokens — all single letters such as *alef*, *bet*, *gimel*, etc. Notably, Qwen’s larger vocabulary also contributes to its much larger *non-Hebrew* allowed set (10,070 digits/punctuation/special tokens vs. 1,874 for Mistral).

3.2 Implementation

Constrained decoding was implemented as a custom `LogitsProcessor` subclass, `HebrewOnlyLogitsProcessor`, registered via HuggingFace’s `LogitsProcessorList` hook that is called immediately before each sampling step.

Mechanism. On the first forward pass the processor builds a Boolean mask of length equal to the actual logit tensor’s vocabulary dimension (which can differ from `tokenizer.vocab_size` due to padding). All positions corresponding to non-allowed token IDs are set to `False`; all positions in the allowed set are set to `True`. At every decoding step the processor sets the logits of all masked-out tokens to $-\infty$, making those tokens impossible to sample. The mask is built lazily and cached, so the overhead is paid only once per generation call.

Decoding configuration. Greedy decoding (`do_sample=False`) was used throughout for determinism. Both constrained and unconstrained runs use identical prompts, formatted with `tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)`, which applies each model’s native conversational format (Qwen: `<|im_start|>user...<|im_end|>`; Mistral: `[INST]...[/INST]`). This ensures a fair comparison: the only variable between the two conditions is the presence or absence of the logits mask.

3.3 Results and Discussion

Output overview. The unconstrained outputs for all 10 queries are fluent, well-structured English responses from both models. The full outputs appear in `decoding_outputs.jsonl`. Table 7 shows the constrained outputs, truncated to 120 characters.

[illegible]

Table 7: Constrained decoding outputs (truncated). Full outputs in `decoding_outputs.jsonl`.

Is the Hebrew text meaningful? No Hebrew characters appear in any constrained output from either model. Despite Hebrew-bearing tokens being present in both allowed sets (3,164 for

Qwen; 38 for Mistral), greedy decoding never selects them.

Does the output relate to the question? No. The constrained outputs contain no semantic content; they are purely structural or punctuation-based symbol sequences with no informational value.

Do the two models differ? The outputs are gibberish for both models, but they follow different degenerate patterns reflecting each model’s internal “schema memory”:

- **Qwen** tends to produce Markdown-like header loops (### **[1. **[[**[[**[** ...) or repeating single symbols (iiii ... , " ,'" ...).
- **Mistral** tends to produce numbered list or hierarchical heading loops (**1. **, **1.1. **, **1.1.1. **, ...), template bracket structures ([**[[**[[** ...), or incrementing number sequences (1-2-3-4-5- ...).

Both patterns suggest the model enters a degenerate attractor state once normal text tokens are unavailable.

Are some languages favoured over others? No natural language is produced. The outputs are language-free sequences of punctuation, digits, and structural tokens.

Root cause. The core problem is a *mismatch between the token constraint and the model’s probability distribution*. When the model is conditioned on an English prompt, its distribution assigns probabilities close to zero to Hebrew tokens and high probabilities to English-letter tokens. Masking out all non-Hebrew-allowed tokens removes the high-probability English tokens, but leaves intact a large set of punctuation and structural tokens that the model assigns moderate probability to. Greedy decoding therefore collapses to whichever punctuation token has the highest logit at each step, quickly entering a repetitive loop.

In principle, adding a Hebrew-language instruction to the prompt (e.g., “Please respond in Hebrew”) would shift the distribution toward Hebrew tokens and likely produce meaningful Hebrew output under the same constraint. However, this was deliberately not done: modifying the prompt for the constrained condition but not the unconstrained condition would make the two conditions incomparable, since any quality difference could be attributed to the prompt change rather than to the token constraint alone. Fine-tuning the model to shift its *prior* distribution toward Hebrew — the subject of Part 4 — is the correct approach.

4 Part 4 - Fine Tuning

4.1 Data

For fine-tuning, I created a supervised instruction dataset of **1,000 English prompt → Hebrew answer pairs**. The prompts were sampled from the `tatsu-lab/alpaca` instruction corpus, and the Hebrew responses were generated automatically using the Gemini API in `code/part4/create_data.py`. The script uses `gemini-3-flash-preview` with the system instruction “Always answer in Hebrew”, then stores each pair in JSONL format.

To prevent data leakage, the 20 assignment evaluation prompts are explicitly listed in the script (`EVAL_INPUTS`) and filtered out before writing training examples. Therefore, none of the evaluation inputs appears in the fine-tuning set.

The final training set covers diverse topics (science, history, writing, everyday reasoning, and basic programming), which helps evaluate whether the model learned a general language behavior (answering in Hebrew) rather than a single narrow template.

4.2 Fine-Tuning Procedure

I fine-tuned `Qwen/Qwen2.5-1.5B-Instruct` using LoRA (PEFT) with `trl SFTTrainer` in Google Colab. LoRA was selected to reduce compute and memory cost while keeping base model weights frozen.

Table 8: LoRA Configuration

Parameter	Value
Rank r	8
α	16
Dropout	0.05
Target modules	<code>q_proj, k_proj, v_proj, o_proj</code>

Table 9: Training Hyper-Parameters

Parameter	Value
Epochs	3
Batch size	2
Gradient accumulation	4 (effective batch size = 8)
Learning rate	2×10^{-4}
Max sequence length	512

I used a completion-only data collator, so loss is computed only on assistant tokens (Hebrew response) while user-prompt tokens are masked with label `-100`. This directly optimizes the target behavior: produce a Hebrew response for an English instruction.

The same Qwen chat template format was used consistently in both training and inference:
`<|im_start|>user ... <|im_end|><|im_start|>assistant ....`

4.3 Evaluation

I evaluated the original and fine-tuned model on all 20 required prompts (stored in `eval_outputs.jsonl`).

Table 10: Evaluation Summary (20 prompts)

Metric	Base model	Fine-tuned model
Responses in Hebrew	0/20 (0%)	20/20 (100%)
Good / Partial / Poor relevance	20 / 0 / 0	3 / 11 / 6

The relevance score was determined using the "LLM as a judge" methodology; the outputs were prompted to a strong LLM (Claude sonnet 4.6), which was required to give a score to each response (Good/Partial/Poor).

Training-loss evidence of learning. The training loss started at **2.825** and decreased to **2.355**. In this setup, loss is the average token-level negative log-likelihood on the assistant response tokens:

$$\mathcal{L} = -\frac{1}{N} \sum_{t=1}^N \log p_{\theta}(y_t \mid x, y_{<t}).$$

A lower value means the model assigns higher probability to the correct next Hebrew token, so the downward trend indicates real learning of the target generation behavior.

Two good examples and one bad example.

- **Good: “What is the capital of France?”** The fine-tuned model answered correctly in Hebrew and identified Paris.
- **Good: “Write a short poem about rain.”** The fine-tuned model produced a coherent Hebrew poem structure, showing it learned the requested output language and style format.
- **Bad: “Name two famous painters from the Renaissance.”** The fine-tuned model hallucinated and listed Shakespeare as a painter, which is factually incorrect.

Per-prompt results (all 20 inputs).

Table 11: Base vs Fine-Tuned Outputs on the 20 Evaluation Prompts

ID	Prompt (short)	Base lang	FT lang	FT quality note
1	Why sky looks blue	English	Hebrew	Poor (topic drift)
2	2 pros/2 cons public transport	English	Hebrew	Partial
3	Email for extension	English	Hebrew	Partial
4	How to make omelette	English	Hebrew	Poor
5	Supervised vs unsupervised	English	Hebrew	Partial
6	Cinderella in 3 sentences	English	Hebrew	Poor
7	Reduce smartphone distraction	English	Hebrew	Partial
8	What happens when water boils	English	Hebrew	Partial
9	Polite refusal to party	English	Hebrew	Partial
10	Practice makes progress advice	English	Hebrew	Poor
11	Capital of France	English	Hebrew	Good
12	Number of continents	English	Hebrew	Good
13	Gravity in simple terms	English	Hebrew	Partial
14	Short poem about rain	English	Hebrew	Good
15	3 exercise benefits	English	Hebrew	Partial
16	How to make tea	English	Hebrew	Partial

ID	Prompt (short)	Base lang	FT lang	FT quality note
17	Noun vs verb difference	English	Hebrew	Partial
18	Renaissance painters	English	Hebrew	Poor (hallucination)
19	Why people dream	English	Hebrew	Partial
20	Advice for a new job	English	Hebrew	Partial

Overall, fine-tuning was **successful for language control** (100% Hebrew outputs) but only **partially successful for factual and instruction accuracy**. The model appears to have learned a strong Hebrew-response prior, while content quality remains mixed on more complex prompts.